BTS CIEL ~ Arduino et Kit Grove

Nom : _____

Initiation à l'utilisation de l' « Arduino Uno » avec un kit de capteurs et d'actionneurs « Grove Starter Kit »

Objectifs :

- Prendre en main une plateforme de prototypage électronique facile à utiliser ainsi qu'un Kit de raccordement rapide de capteurs et d'actionneurs Grove.
- Apprendre le développement logiciel dans un contexte de système embarqué.

Prérequis :

- Connaître les bases de numération et codage
- Savoir utiliser un multimètre

Outils mis à disposition :

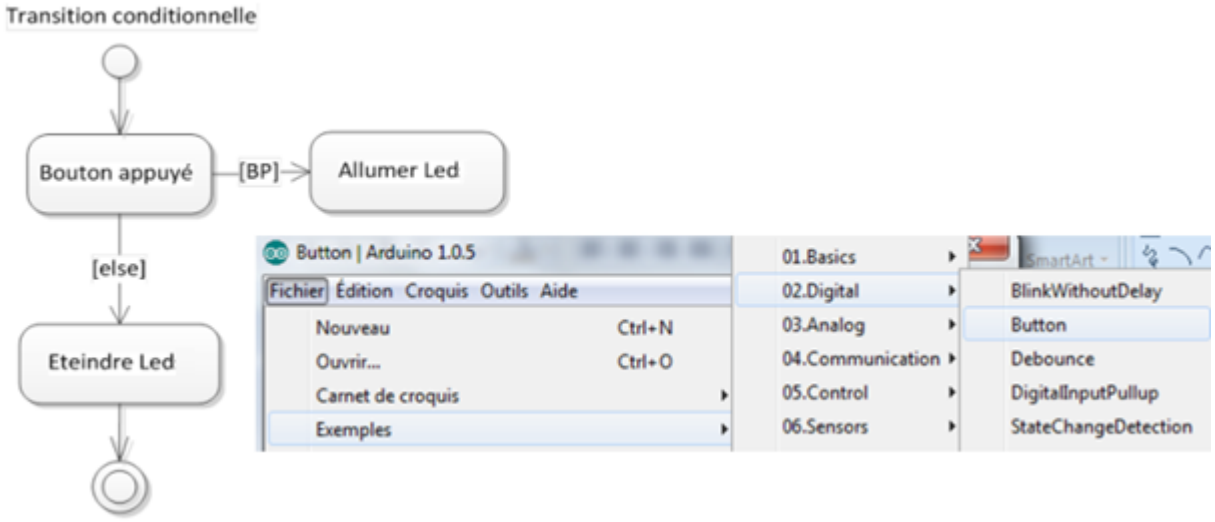
- 1 ordinateur équipé de l'IDE arduino
- 1 carte « Arduino Uno » + câble USB type A/B
- 1 kit « Grove Starter »
- 1 multimètre
- 2 résistances de 220 ou 330 ohms
- 1 breadboard et ses fils
- 1 oscilloscope

- Vérifier le contenu du Kit à chaque prise en main et à chaque entreposage.
- ⚠ Signaler les éléments manquants à votre enseignant.

Gestion d'une entrée et d'une sortie logique (bouton poussoir + une LED)

Objectifs :

- Configurer une broche en entrée logique
- Lire l'état d'un bouton
- Provoquer l'allumage de la LED grove « Bleue » en fonction de l'état logique de l'entrée logique



➲ Réaliser le montage attendu, ouvrez l'exemple de code « Digital > Button » et modifier le programme pour atteindre un bon fonctionnement

➲ Enregistrer votre programme sous le nom « logicalInToLogicalOut.ino » dans votre dossier personnel

Un commentaire du code exemple indique qu'une résistance de 10 kΩ doit être placée en série avec le bouton poussoir…

```
 The circuit:
  - LED attached from pin 13 to ground through 220 ohm resistor
  - pushbutton attached to pin 2 from +5V
  - 10K resistor attached to pin 2 from ground
```

➲ Vérifier sur le schéma du bouton poussoir Grove si cette résistance a bien été ajoutée

Q1. Copier le schéma du module « Bouton Grove » et indiquer le nom de la résistance utilisée si elle a été trouvée et sa valeur.

R1.

Q2. Coller la copie d'écran du code « Button » modifié (y compris les commentaires) et commenté par vos soins…

R2.

Gestion d'une entrée et de plusieurs sorties logiques (bouton poussoir + 3 LEDs dont 2 sur breadboard)

Vous l'avez constaté, vous n'avez qu'un seul socket Grove LED dans le kit…

Objectifs :

- Vous devez ajouter les deux autres LEDs (rouge et verte) du Kit dans votre montage au montage précédent.
- Vous devrez utiliser une breadboard et deux résistances de 220 Ω ou 330 Ω.
- A l'appui sur le bouton poussoir, les 3 LEDs devront s'allumer, elles devront s'éteindre au relâchement du bouton.
- Chaque LED devra utiliser une sortie logique différente des autres LEDs

➲ ⚠ Débrancher les sources d'alimentation de l'Arduino avant de procéder au montage (prise USB).
➲ Par la suite, il ne faudra pas oublier de faire vérifier vos premiers montages par votre enseignant avant de rebrancher la prise USB
➲ S'aider de l'annexe « Caractéristique LEDs » en fin de document.

Q3. Les LEDs Rouge, Verte et Bleue ont-elles toutes la même tension de fonctionnement pour un courant identique (ex.20mA) ?

R3.

Q4. Calculer les valeurs de résistances Rr, Rg et Rb à associer à chacune des LEDs.

R4.

Q5. Qu'est-ce-qui se passerait si on branchait une LED sans résistance en série sur une sortie d'un Arduino ?

R5.

➲ A l'aide de l'application « fritzing », réaliser un schéma des modifications à apporter avec la breadboard, la LED rouge et la verte, l'Arduino Uno et les câbles.

Q6. Coller votre schéma ci-après

R6.

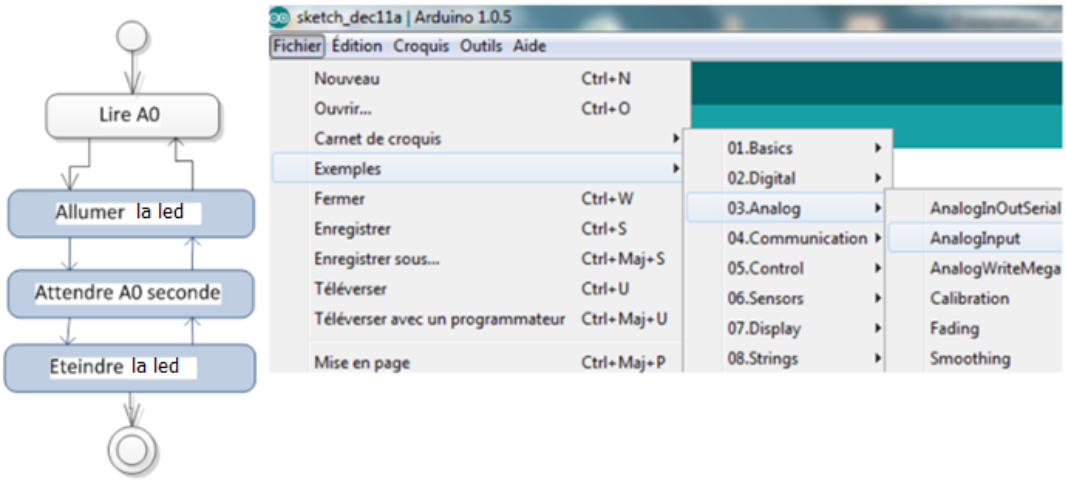
➲ Montrer votre schéma à votre enseignant
➲ Réaliser votre schéma sur la breadboard
➲ Faire vérifier vos premiers montages par votre enseignant avant de rebrancher la prise USB.

➲ Adapter et tester le programme pour qu'il réponde aux attentes énoncées
➲ Enregistrer votre programme sous le nom « testBreadboard.ino » dans votre dossier personnel

Gestion d'une entrée analogique (potentiomètre) pour piloter une sortie logique (LED)

Objectifs :

- Lire une entrée analogique sur laquelle un potentiomètre a été raccordé
- Faire clignoter la LED Grove Bleue avec une fréquence variable en fonction de la position du potentiomètre.
- Se familiariser avec la notion de conversion analogique/numérique (résolution/quantum)



- Ouvrir le programme d'exemple AnalogInput
- Analyser le programme
- Adapter le programme pour faire clignoter la led à une fréquence variable en fonction de la position du potentiomètre.

Q7. Quelle modification avez-vous apportée au code exemple ? (indiquer juste la modification apportée)

R7.

Q8. Quelle est la plage de valeurs possibles qui peuvent être attribuées à la variable « sensorValue » lors de cette instruction sensorValue = analogRead(sensorPin); ? Expliquez-en la raison.

R8.

Q9. Quelle est l'unité de temps utilisée par la fonction delay() dans l'instruction « delay(sensorValue); » ?

R9.

On souhaite pouvoir faire clignoter la LED beaucoup plus lentement

➲ Utiliser la fonction map() pour permettre de fixer le délai après l'allumage et l'extinction entre **250 ms et 5 secondes**.

https://www.arduino.cc/reference/en/language/functions/math/map/

Exemple de code d'utilisation de la fonction « map() »

```
/* Map an analog value to 8 bits (0 to 255) */
void setup() {}

void loop()
{
  int val = analogRead(0);
  val = map(val, 0, 1023, 0, 255);
  analogWrite(9, val);
}
```

Définition de la fonction « map() »

```
long map(long x, long in_min, long in_max, long out_min, long out_max)
{
  return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}
```

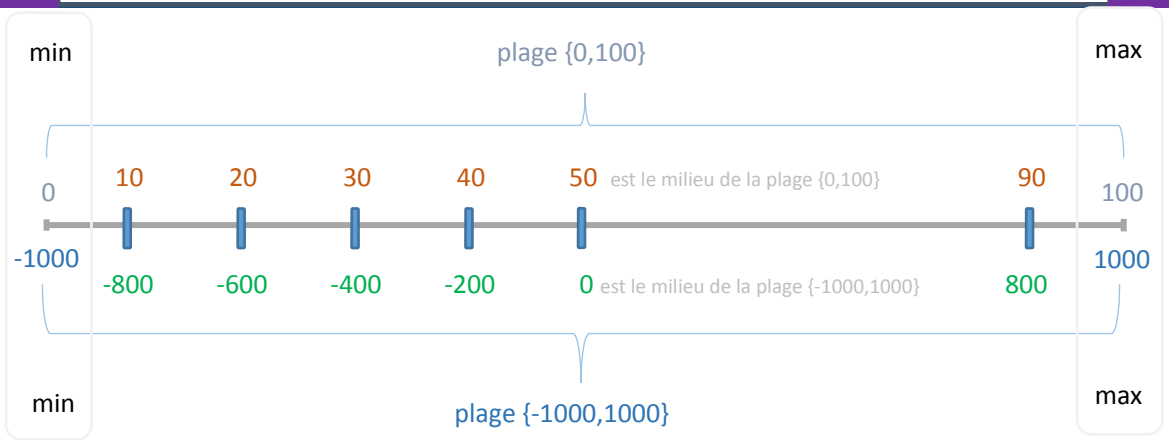
	plage min	plage max
input	**0**	**100**
output	**-1000**	**1000**

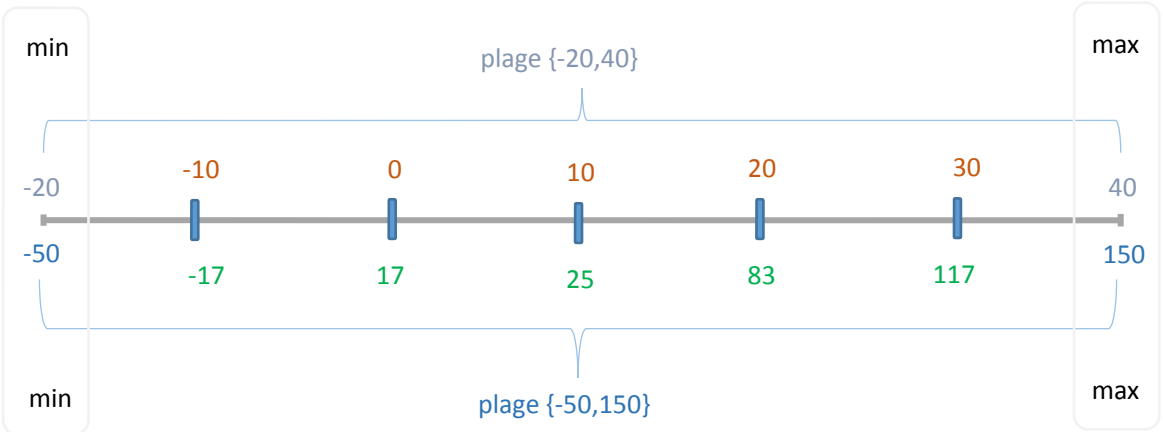
x	**40**
map()	**-200**

(x-in_min)*(out_max-out_min)/(in_max-in_min)+out_min

simulation de la fonction map()

faire un double clic sur cette zone puis
saisir les valeurs dans les cellules bleues





Exemples de transposition linéaire réalisable avec la fonction map()

Les principaux types de variables Arduino :

Il existe différents types de variables pour différents types de données, en voici quelques-uns :

- char : pour stocker des caractères (taille : un byte).
- byte : pour stocker un chiffre compris entre 0 et 255.
- int : pour stocker un chiffre compris entre 2-15 et 215-1, c'est-à-dire, entre -32,768 et 32,767 (taille : 2 bytes (16 bits)).
- unsigned int : n'a pas de signe donc peut stocker des valeurs comprises entre 0 et 216-1, c'est-à-dire entre 0 et 65,535 (taille : 2 bytes).

- long : pour des valeurs comprises entre -2,147,483,648 et 2,147,483,647 (taille : 32 bits (4 bytes)).
- unsigned long.
- float : pour nombres décimaux compris entre -3.4028235E+38 et +3.4028235E+38 (taille : 32 bits (4 bytes)).
- double : pour nombres décimaux aussi (taille : 8 bytes (64 bits)).

Q10. Quel type de variable est retourné par la fonction « map() » ?
R10.
Q11. Donner la description de ce type de variable (nombre de bits et autres informations utiles).
R11.
Q12. Expliquer le fonctionnement et l'utilité générale de la fonction map().
R12.
➲ Modifier le programme en utilisant la fonction map() et tester le jusqu'à atteindre l'objectif énoncé.
Q13. Coller une copie d'écran du code obtenu (sans les commentaires du début)
R13.
➲ Enregistrer votre programme sous le nom « potarToLedBlink.ino » dans votre dossier personnel
➲ Positionner le potentiomètre pour obtenir un délai de 5 secondes d'allumage puis 5 secondes d'extinction de la LED.
➲ Juste après l'allumage de la LED, faire varier brusquement la position du potentiomètre pour faire diminuer le délai de clignotement.
Q14. Est-ce que la diminution du délai est prise en compte instantanément ou faut-il attendre la boucle suivante ? Expliquer pourquoi…
R14.

Production de son (buzzer)

Objectifs :

- Utiliser les fonctions tone() et noTone()
- Travailler la notion de fréquences sonores
- Produire une mélodie en réponse à une action

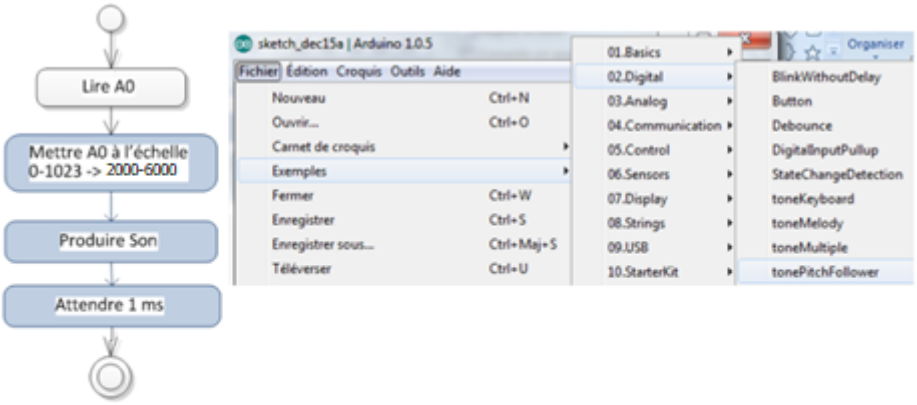
➲ Ouvrir et tester le programme Buzzer-Digital-Write (Ressources\Exemples)
➲ Ouvrir et tester le programme Buzzer-Tone (Ressources\Exemples)

Q15. Quelles sont les fonctions utilisées par les programmes pour générer une note sur la sortie logique utilisée ?
R15.
https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/
➲ Modifier les 2 programmes (Buzzer-Digital-Write et Buzzer-Tone) pour qu'ils génèrent la mélodie suivante lorsqu'on appuie sur le bouton poussoir :
« do - do - do - ré - mi - ré - do - mi - ré - ré - do »
NB. Pour ceux qui ne sont pas mélomanes : C=do, D=ré, E=mi, F=fa, G=sol, A=la, B=si, c = do mineur, C = do majeur Cs signifie do dièze (sharp # en anglais)…
Q16. Donner le titre de la mélodie que vous avez programmée.
R16.
➲ Enregistrer les 2 programmes (Buzzer-Digital-Write-Nom-Mélodie.ino et Buzzer-Tone-Nom-Mélodie.ino) dans votre espace de sauvegarde personnel

Gestion d'une entrée analogique (potentiomètre), génération d'un son (buzzer) et transmission de données par liaison série

Objectifs :

- Produire un son de fréquence variable en fonction de la lecture sur une entrée analogique

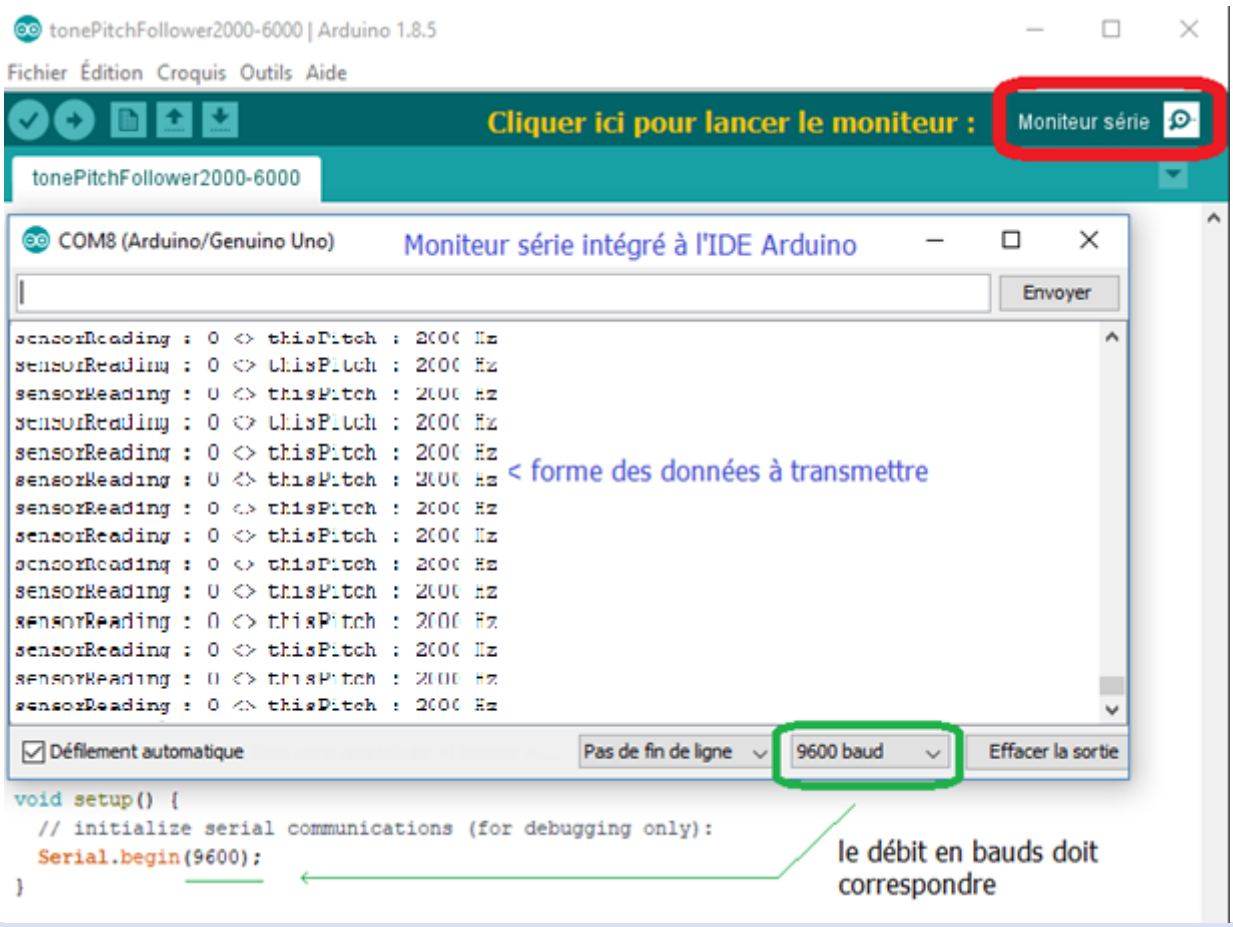


- Ouvrir le programme d'exemple tonePitchFollower
- Analyser le programme
- Adapter le programme pour faire sonner le buzzer à fréquence variable en fonction de la position du potentiomètre entre 2000 et 6000 Hz.

➲ Modifier le programme pour faire afficher « sensorReading : xxxx <> thisPitch : yyyy Hz » dans le moniteur série ou xxxx représente la valeur issue de la conversion anlogique numérique et yyyy la fréquence en Hz

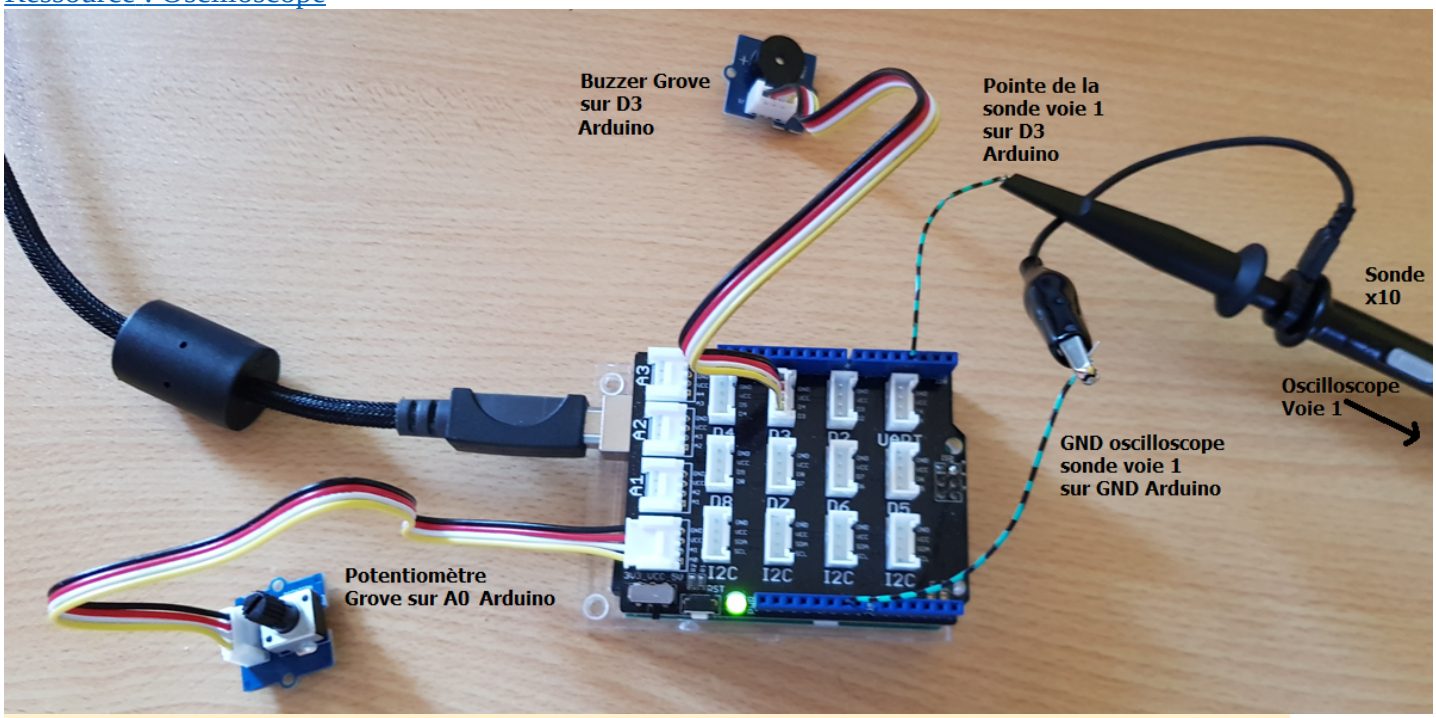
Nb. la fonction print() pourra être utilisée pour éviter un retour à la ligne. Chercher la différence entre les deux fonctions sur le site de référence :
arduino.cc/reference/en/language/functions/communication/serial/
Un moniteur série est intégré à l'IDE Arduino.

➲ Enregistrer votre programme sous le nom « analogInToBuzzer.ino » dans votre dossier personnel



➲ Utiliser un oscilloscope pour visualiser le signal généré sur la sortie D8

[Ressource : Oscilloscope]



Q17. Quelle est la tension d'entrée maximale de l'oscilloscope utilisé ?

R17.

Q18. Quelle est la forme du signal observé ?

R18.

Q19. Donner la tension mini et la tension maxi visualisées à l'oscilloscope.

R19.

➲ Activer la mesure de la fréquence sur l'oscilloscope (Meas > Type : Fréq)

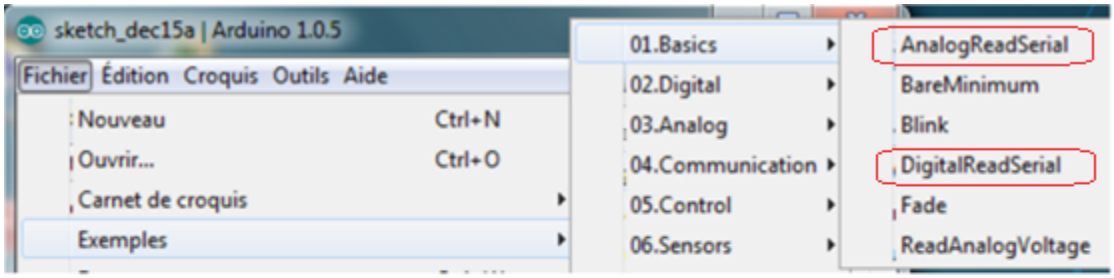
Q20. Quel paramètre fait-on varier sur la sortie lorsque le potentiomètre modifie la consigne en entrée ?

R20.
Q21. L'écart de mesure à l'oscilloscope de la fréquence du signal est-il inférieur à 1% ?
R21.
Q22. Faire une copie de l'écran de l'oscilloscope…
R22.
Q23. Que pouvez-vous conclure sur la qualité du signal électrique généré sur la sortie D8?
R23.

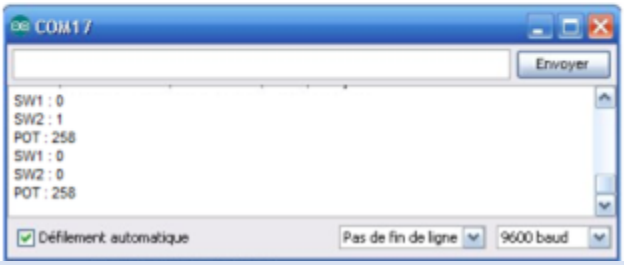
Gestion d'une liaison série pour transmission de données acquises (potentiomètre + bouton poussoir + capteur tactile)

Objectifs :

- Initialiser l'objet Serial
- Ecrire des données correspondant aux états de broches logiques ou analogiques sur le port série
- Utiliser le moniteur série de l'EDI Arduino
- Lire les données transmises dans le moniteur série intégré à l'IDE Arduino



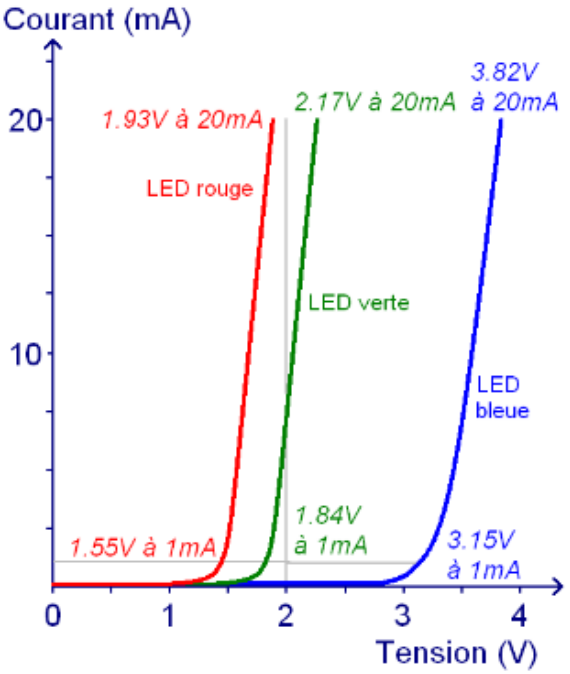
- Ouvrir les programmes d'exemple AnalogReadSerial et DigitalReadSerial
- Analyser les programmes
- Adapter un programme pour transférer toute les 500ms l'état du bouton poussoir (SW1), celui du capteur tactile (SW2) et la position du potentiomètre



➲ Enregistrer votre programme sous le nom « serial9600.ino » dans votre dossier personnel
➲ Utiliser l'appication Putty (disponible dans le dossier Ressources/Applications) pour afficher les données transmises par l'Arduino

Q24. Faire une copie d'écran de la configuration de Putty qui permet le dialogue avec le port série de l'Arduino.
R24.
Q25. Faire une copie d'écran des données transmises par l'Arduino et réceptionnées par Putty
R25.

Annexe : Caractéristiques LED



Couleur	Longueur d'onde (en nm)	Tension de seuil
Blanc		3,5V
Bleu	450 - 500	2,5V à 2,8V
Infrarouge	>760	1,6V
Jaune	585 - 590	2,1V
Orange	600 - 620	2V
Rouge	625 - 655	1,6V à 2V
Ultraviolet	<400	3,1V
Vert	525 - 565	2,1V à 2,5V
Violet	400 - 450	2,7V à 3,1V

Exemple - Tensions de seuil pour des LEDs en fonction de la couleur

